

Learning Dexterous Manipulation:

Technical Supplement

Luai Abuelsamen

This supplement preserves the detailed evidence behind three compact parts of the survey: the paper-code comparisons, the coding of earlier surveys, and the statistical calculations behind the proposed evaluation protocol. It is separated because these records support verification but do not form part of the paper’s narrative.

APPENDIX A

PAPER AGAINST RELEASED CODE, IN FULL

Each entry below is the disagreement text stored in the row, unedited, and the class beside it is the one Section V-F and Section VIII count. A *contradiction* is a paper stating one value where the shipped code demonstrably states another. A *parse-limitation* is a comparison withdrawn because this survey’s own parse could not settle it. *Code-absent* means the described component is not in the released repository, *version-skew* that the repository is a later generation than the paper, and *internal-inconsistency* that the paper disagrees with itself with no code implicated. Where a comparison was narrowed or withdrawn under review, the review note is printed beneath it. Every *contradiction* entry also carries an *Artefact* line: the repository, the commit `corpus/code_manifest.json` records, the file inside it and where in that file the value sits, so the entry can be settled without asking anyone. None of those authors was written to before this survey was posted. Section V-F says so beside the finding, sets out what a fetched commit can and cannot show, and states the route by which a disputed entry is corrected; the letters that were drafted and not sent are in `outreach/` in the corpus, which is available from the author at the address in the author block. The reward-term extraction these rows were read out of is in the repository rather than here, with the rest of the tabulation Appendix A describes.

contradiction, 9 rows.

- PDDM [1] (high). Table 2 states `obs-dim 46` for In-hand Reorientation while the released `cube_env.py` code sums to 39; the Baoding reward code includes an extra `-10*wrst_too_high` term absent from Table 2’s printed formula.
Artefact: `google-research/pddm` at `06b88cdbaf`, `pddm/envs/cube/cube_env.py` (`_get_obs`).
- DexPoint [2] (high). The released code’s reward adds several terms absent from the paper’s four-term Eq. 5 (a lift-threshold bonus, a target-distance term, a rotation bonus, and an IK controller-tracking penalty) and reshapes the reach/lift terms into inverse-distance and clipped forms rather than the paper’s plain distance/height-difference formulas.

Artefact: `yzqin/dexpoint-release`
at `17f1e238bb`, `dexpoint/`

`env/rl_env/relocate_env.py`
(`AllegroRelocateREnv.get_reward`).

- DeXtreme [3] (high). Action Delta Penalty weight is `-0.25` in paper Table 2 but `-0.2` in the ADR yaml and `-0.01` in the ManualDR yaml; the Joint Velocity Penalty in code normalises velocity by `(max_velocity-vel_tolerance)` unlike the paper’s stated formula; code has a `timeout_rew` term absent from the paper’s reward table; Appendix Table 12 states critic learning rate `5e-4` and KL threshold `0.16`, vs body text/code values of `5e-5` and `0.016`.

Artefact: `isaac-sim/IsaacGymEnvs`
at `aed298638`, `isaacgymenvs/cfg/task/AllegroHandDextremeADR.yaml`
(`actionDeltaPenaltyScale`).

- Visual Dexterity [4] (high). Table S1 states 32000 teacher training environments, but the released config sets `alg.num_envs` to 8000 (parent config 16384); `fallDistance` differs across two shipped configs (`0.24` vs `0.15`, only the latter matching Table S1’s threshold); the paper’s Eq 8 penultimate-joint penalty (`c7=-2`) does not appear anywhere in the released reward code; the config carries a dead `distRewardScale=-10.0` key never used in `compute_reward`; and the paper’s table-friction lower bound (`0.05`) differs by a factor of 10 from the code’s randomized lower bound (`0.005`).

Artefact: `Improbable-AI/dexenv` at `ad9634e9d2`, `dexenv/conf/dclaw.yaml` (`alg.num_envs`).

- DexPBT [5] (high). Paper presents the reward as 4 mutually exclusive stage terms (`r_reach`, `r_pick`, `r_targ`, `-r_vel`), but code’s `compute_kuka_reward` sums 8 named components, one of which (`hand_delta_penalty`) is multiplied by 0 and disabled; there is no single `r_vel` term in code, instead separate `kuka/allegro` action penalties whose exact formula is not shown. Also, the paper reports zero experiments with domain randomization, yet the shipped `AllegroKuka.yaml` already carries a fully specified DR schedule (disabled via `randomize: False`).

Artefact: `NVIDIA-Omniverse/IsaacGymEnvs` at `aed298638`, `isaacgymenvs/tasks/allegro_kuka/allegro_kuka_base.py` (`compute_kuka_reward`).

Review: R3 adversarial review: the disabled-randomisation half is withdrawn, since the note finds it consistent with the paper; the zeroed reward term stands

- PhysHOI [6] (high). The released `compute_humanoid_reward` sets the body position-velocity error and the object rotation and rotation-velocity errors to `zeros_like`, unconditionally, with the computation that would produce them commented out on the same lines, while Table 4

lists nonzero $\lambda^{or}=0.1/\lambda^{orv}=0.01$ weights for GRAB: in that file the object’s orientation error is the constant zero and those weights cannot change the reward.

Artefact: wyhuai/PhysHOI at 6095c605e2, physhoi/env/tasks/physhoi.py (compute_humanoid_reward).

- UniDexGrasp [7] (high). The paper describes a four-term weighted reward ($r_{goal} + r_{reach} + r_{lift} + r_{move}$ via Table 7’s omega weights) but the released compute_hand_reward implements a different threshold-gated torch.where cascade with distinct hardcoded coefficients that do not map one-to-one onto the paper’s weights.

Artefact: PKU-EPIC/UniDexGrasp at 36c9bfcf7c, dexgrasp_policy/dexgrasp/tasks/shadow_hand_grasp.py (compute_hand_reward, goal_cond branch).

- PenSpin [8] (medium). The appendix states a randomised disturbance force and the released task config sets its scale to zero. A second half of the original comparison, that the released code disables the paper’s tactile observation channel, is withdrawn: the config read has 96 observation dimensions and enable_tactile: False, consistent with the proprioception-only student policy rather than the tactile-and-point-cloud oracle, and the paper never claims the student has tactile input. The code’s reward scale-key names (e.g. rotate_reward_scale, pencil_z_dist_penalty_scale) also do not 1:1 name-match the paper’s Table 4 weight list, though matched values agree.

Artefact: HaozhiQi/penspin at 5035c52dc9, configs/task/AllegroHandHora.yaml (forceScale).

Review: Narrowed at the point of drafting the letter to its authors, which was never sent. The tactile-channel half is withdrawn as a plausible reading of which pipeline stage the config belongs to; the disturbance-force half stands. Held at medium confidence pending a direct code read.

- PianoMime [9] (high). Paper’s Table 3 states 2 weighted reward terms (Key Press 2/3, Mimic 1/3), but the released code sums roughly 5 unweighted terms (key press doubled, sustain, energy and fingering hardcoded to return 0, forearm-collision) plus a separately-added mimic wrapper term.

Artefact: sNiper-Qian/pianomime at c4abefac8d, single_task/piano_with_shadow_hands_res.py (_set_rewards).

internal-inconsistency, 4 rows.

- ALOHA [10] (high). Algorithm 1 pseudocode states $L_{reconst} = \text{MSE}$, but Sec.IV.C’s prose explicitly states L_1 loss is used instead; the algorithm box and the implementation text disagree; code/md captures only function signatures, not bodies, for policy.py’s loss implementation
- 3D Diffusion Policy [11] (medium). the paper’s prose states the network predicts the noise added to the data, but the shipped default config trains with prediction_type: sample (predicting the denoised action a^0 directly), not epsilon; the paper only qualifies this later in the same

section (Fig. 7 ablates both).

- OmniH2O [12] (medium). stumble weight -0.00125 (paper) vs -1250 (code); max-feet-height sign/magnitude differ (+1000 paper vs -2500 code, a penalty not a bonus); paper’s $\exp(-c*///)$ form vs code’s $\exp(-\text{err}^2/\text{sigma})$; curriculum level-down threshold 40 (paper) vs 50 (code). *Review:* Withdrawn as a contradiction at the point of drafting the letter to its authors, which was never sent. Four of the five weight comparisons match the paper’s table to the digit once a systematic $\times 1.25$ curriculum factor is accounted for, and only the stumble weight differs, by a factor of about a million, which is a typo signature in the paper’s own table rather than evidence of a different trained objective. The hands in this work are also driven open-loop from VR pose, outside the policy and outside the reward, making it a poor fit for a dexterous-manipulation reward census regardless.
- DexMachina [13] (low). paper describes a plain weighted sum $\text{lambda_task} * r_{\text{task}} + \text{lambda_imi} * r_{\text{imi}} + \text{lambda_bc} * r_{\text{bc}} + \text{lambda_con} * r_{\text{con}}$ with unspecified weights; code implements a multiplicative task term with per-component beta decay, an unmentioned 0.1 force-penalty term, and curriculum-driven decay of auxiliary weights not described as such in the paper *Review:* R3 adversarial review: the multiplicative form credited to the code is printed in the paper itself

version-skew, 4 rows.

- Hora [14] (high). The parsed code commit’s joint-noise range, disabled default disturbance force, and default cube object type differ from the paper’s stated values; the code README itself says to use tag v0.0.1, not the parsed commit, to reproduce paper numbers.
- π_0 [15] (high). The released repo (openpi, commit 215abfb2) has evolved past this paper: it also documents pi0-FAST (autoregressive) and pi0.5 checkpoints/configs not described in this paper, which describes only the flow-matching pi0 and the non-VLM pi0-small ablation.
- GR00T N1.6 [16] (high). code/md is the current Isaac-GR00T main branch (N1.7 generation, commit 51d4c89f), not the N1.6 checkpoint; the repo names a separate n1d6 branch for N1.6 that was not fetched. The page states N1.6’s backbone is “an internal NVIDIA Cosmos-2B VLM variant,” but the repo’s own changelog states the Eagle backbone (nvidia/Eagle-Block2A-2B-v2) was used through N1.6 and only replaced by Cosmos-Reason2-2B in N1.7 – whether the page’s Cosmos-2B variant is a third distinct model or the page is describing N1.7 under an N1.6 headline is not resolvable from what was fetched. *Review:* The page attributes a Cosmos backbone to N1.6 while the repository changelog says N1.6 used Eagle and N1.7 made the switch, so the repository is a later generation than the page describes.
- GR00T N1 [17] (high). The parsed code repo (Isaac-GR00T, commit 51d4c89f) is a later N1.7 generation, not the paper’s GR00T-N1-2B: VLM backbone changed from the paper’s Eagle-2 to Cosmos-Reason2-2B (via Qwen3-VL); action horizon expanded from the paper’s stated

H=16 to 40; state/action dimensions expanded from 29 to 132; DiT layers changed from 32 to 16; embodiment scope widened to include Unitree G1, AgiBot G1, and a ‘YAM’ arm not described in the paper.

code-absent, 8 rows.

- AsymDex [18] (medium). Code contains a GraspAndPlace-task reward variant (AllegroHandDualArmGraspAndPlaceAsymDex.py) with exponential hand-distance and block-alignment terms not one of the four BiDexHands tasks described in the paper body.
- DexMimicGen [19] (high). The environment code includes an unused ‘reward_shaping’ branch (if self.reward_shaping: pass) that is never described or used in the paper text; only the sparse binary completion reward is used, as the paper states.
- DrEureka [20] (high). The released DrEureka repo (code/md) contains only forward_locomotion/ and globe_walking/ trees; the LEAP-hand cube-rotation training/deployment code and the quoted Prompt-13 reward are the LEAP-hand authors’ own code, not included in the released repo, so the cube-rotation reward cannot be cross-checked against a code file.
- Open-TeleVision [21] (low). The paper’s prose states 25k training iterations, lr 5e-5, batch size 45, but the code repo’s example training command instead shows 50000 epochs and an explicit kl_weight=10 not mentioned anywhere in the paper text.
Review: R3 adversarial review: a behaviour-cloning paper with no reward; it compared paper iterations to a README example’s epochs
- Being-H0 [22] (high). The actual downstream action-head module referenced in §4.3 (learnable query tokens, proprioceptive projector f_p , regression head f_r) is not present in the parsed code; the repo’s own TODO list marks ‘Training code and scripts’ as unreleased, so the action-chunk-length=16 value and exact proprioception-vector fields come only from inference/eval CLI examples, not a verifiable implementation.
- $\pi_{0.5}$ [23] (high). The paper describes a hybrid discrete-FAST-token-pretrain-then-flow-matching-post-train recipe (Eq. 1, alpha: 0 to 10), but the released repo states plainly it ‘currently only support[s] the flow matching head for both $\pi_{0.5}$ training and inference.’
- $\pi_{0.6}^*$ [24] (high). The general openpi framework ($\pi_0/\pi_{0.5}$ /FAST, flow-matching action head, ALOHA/DROID/LIBERO adapters) is released at commit 215abfb2, but the RECAP-specific pieces described in the paper – the advantage-conditioning input, the distributional value function, the RECAP training loop (Algorithm 1), and the espresso/laundry/box-assembly checkpoints and datasets – are not present in the parsed repo.
- Being-H0.5 [25] (high). Released code (commit e12ac44f) exposes only benchmark/inference/config scaffolding (policy.py, beingh_policy.py, dataset-transform configs). no MoT/MoF model definition, no rectified-flow action-expert source, and no config for any dexterous-hand

embodiment; the only post-train configs present (libero_robocasa.yaml, libero_all.yaml, robocasa_human.yaml, so101_example.yaml) are all gripper-only, so the dexterous-hand claims cannot be cross-checked against released code.

parse-limitation, 13 rows.

- ArtiGrasp [26] (low). paper’s two regulariser weights ($w_{rh}=0.5$, $w_{ro}=0.2$) correspond to five separate velocity penalties in code (-0.5,-0.2,-0.5,-0.5,-0.3); the fingertip weight 12.0 and lambda 5.0 in Table 6 do not appear in the yaml; reward weights differ between curriculum phase-1 and phase-2 configs though Table 6 reports only one set
Review: R3 adversarial review: the reward body is C++ and absent from the parse; the weight split is the paper’s documented curriculum
- Diffusion Policy [27] (high). the paper’s core loss equations (DDPM training loss, EBM/InfoNCE, score-matching) are unrecovered images in the parsed text, and code/md is a signature-only API map, so compute_loss’s body could not be verified from code; only the exact shipped noise-scheduler config (DDPMScheduler, 100 train steps) is confirmed.
- Eureka [28] (low). The released repo’s default eureka/cfg/config.yaml ships iteration:1, sample:3 (a fast-test default), not the paper’s reported experimental configuration of $N=5$ iterations $\times$ $K=16$ samples $\times$ 5 independent runs, and LLM-generated reward code itself is never checked into the repo (written at runtime to a gitignored outputs/ directory).
Review: R3 adversarial review: the repo README documents the paper’s defaults; the cited config block is a fast-test preset
- UniDexGrasp++ [29] (low). The paper’s four named reward weights (0.5, 0.1, 2, 10) numerically match literals in the released compute_hand_reward, but the code never labels them with the paper’s variable names, and the paper’s own display equations were dropped from the PDF, so the correspondence is inferred rather than confirmed by either source.
- BiDexHD [30] (high). Table 5 weights (w_r , w_t , w_l , w_4 , $\lambda_w=0.12$, $\lambda_{ft}=0.48$) cannot be matched to code reward lines because the code md truncates the reward function before stage-2/final sum; λ_w and λ_{ft} numerically match the code’s grasp-condition thresholds (0.12, 0.12x4 fingers) rather than confirmed reward weights
- GraspXL [31] (low). The code splits the paper’s single $r_{reg} = -w_h/u_h/2 - w_o/u_o/2$ into four separately-weighted terms (wrist vel, wrist qvel, object vel, object qvel), and the floating-phase object-velocity coefficient (-1.5) does not match the paper’s stated $w_o=0.1$, so the regularization weighting cannot be reconciled 1:1 with Table 9.
Review: R3 adversarial review: the environment source is not in the parse and the note says the reward cannot be verified

- HumanPlus [32] (medium). code implements a superset of legged_gym reward terms (lin_vel_z, ang_vel_xy, orientation, base_height, torques, dof_vel, dof_acc, action_rate, collision, termination, dof/torque limits, tracking_lin/ang_vel, feet_air_time, stumble, stand_still, feet_contact_forces, target_jt) beyond the 8 terms documented in the paper’s Table 1
- Omnigrasp [33] (medium). The code’s compute_pregrasp_reward_time hard-codes w_pos, w_rot = 0.9, 0.1, silently overriding the rwd_specs weight arguments passed into the function; separately, the paper’s pre-grasp reward weights (w_hp, w_hr) are not resolvable from either the paper text or Appendix Table 7.
- ResDex [34] (medium). The base-policy and residual-policy task files implement different variants of compute_hand_reward (whether finger/hand-distance terms are included alongside goal_hand_rew/hand_up/bonus differs), a difference the paper’s Sec 4.1/4.4 text does not describe, and the released configs’ env count (4096 default) and iteration counts (2500/5000) do not match the paper’s stated 11,000 envs and 5,000/20,000 iterations.
- DexGraspVLA [35] (high). Hardware-interfacing code is explicitly withheld (“Due to intellectual property constraints, we are unable to open-source the hardware-related code”); only function signatures for the controller loss (compute_loss, noise_assignment) are present in code/md, not their bodies, so the MSE/Immiscible-Diffusion loss cannot be verified against code.
- DexTrack [36] (low). The paper gives one canonical reward-weight table (Table 3), but the released repo’s task-config YAMLS contain several divergent, unreconciled reward-coefficient sets across AllegroHandTracking/...TrackingGeneralist/other variants, and it is not identifiable from the parsed source which config produced the paper’s headline Table 1 numbers.
- DexVLA [37] (high). code/md only carries class/function signatures, not bodies, for the diffusion head (modeling_scaledp.py) and no config file with the α weight was captured; the paper’s diffusion loss and denoising process cannot be verified against code bodies. only the multi-head ScaleDP signature and a DDIM fp32 patch are available, so none of the paper’s tables can be reproduced from what was parsed.
- ManipTrans [38] (low). paper’s PPO learning rate (5e-4) and env count (4096) differ from the shipped README/yaml defaults (2e-4, 8192)
Review: R3 adversarial review: the cited values are a README override and an unused fallback; the repo’s own config matches the paper

APPENDIX B

THE EXISTING SURVEYS, AND WHAT EACH COVERS

Fourteen corpus entries are themselves surveys or engine-comparison studies. Table I in Section I sets them on one set of columns, three levels to a cell, recording what each work covers and not how well. This appendix is the prose behind those levels: each glyph in that table coarsens one of

the sentences below. Section I states what this survey adds over them and does not repeat it here.

The survey of An et al. [39] is the closest in subject, covering imitation learning for multi-fingered hands by learning family, end-effector class and demonstration source. It gives reinforcement learning no taxonomy, treats bimanual work as a single “multi-agent” subsection, compares no simulator, and defines no evaluation metric. The survey of Welte et al. [40] finds only seven dexterous works that use interactive imitation learning and carries a fifteen-hand commercial table, with no bimanual section, no benchmark table and no contact modelling. The survey of Zhao et al. [41] supplies the standard sim-to-real split and predates GPU-parallel simulation. The survey of Firoozi et al. [42] has zero occurrences of bimanual, tactile or in-hand.

The survey of Bai et al. [43] spans all of manipulation across 212 pages. Its Sec. 4.3 on dexterous manipulation runs about 720 words, the third longest of its ten task subsections behind grasping and quadrupedal manipulation, which is a real treatment and not a passing mention. The difference is elsewhere. Its Sec. 1.2 lists dexterous manipulation among the topics that “existing surveys” cover from “narrower perspectives” and defers it to two of them.

The survey of Zhao et al. [44] is the most recent hand-centred one, with a 29-hand anatomy table and a task-by-paradigm taxonomy. Its Sec. IV-C names the reference-versus-rollout split that Section I credits it with, and its Sec. III-F is the one bimanual subsection, 16 cited works with no coordination analysis.

The review of Kaup et al. [45] is the predecessor closest to the engine comparison, and it reviews Brax, Chrono, Gazebo, MuJoCo, ODE, PhysX, PyBullet, Unity and Webots for reinforcement learning research. It scores each on documentation, model and environment creation, URDF and MJCF support, and readiness for multi-agent work. It runs no benchmark of its own and says so in its Sec. V, that implementing the same scenarios across nine engines “goes beyond the scope of this paper”. Its running bodies are ant-and-humanoid RL benchmarks rather than hands, and it discusses no timestep, no friction model, no contact formulation and no penetration.

The oldest rows carry the cost of the corpus. Okamura et al. and Piazza et al. [46], [47], and Roa and Suárez [48], are behind publisher paywalls with no author-hosted copy found on 2026-09-18, and Bicchi [49] is in the same position with no row at all. Ma and Dollar [50] are a different case. The fetch that failed when its note was written succeeded afterwards, so a seven-page PDF is on disk with a recorded hash, and no note has been read from it. All five are cited by metadata only and nothing in this survey describes their contents. The open chapter [51] overlaps the paywalled Bicchi paper without being identical to it, so it is quoted in its own right.

APPENDIX C

WHERE THE PROTOCOL’S COUNTS COME FROM

Every count in Table IV is derived below, and each axis is derived for the statistic that axis actually reports rather than by one convention applied to all of them: a single rate takes a

Wilson half-width, a matched comparison takes McNemar, a ratio takes the standard error of the log ratio, and a correlation takes the Fisher-z interval. Section VII-C states what these derivations conclude. This survey re-ran no method, so every count here rests on an interval width, a power calculation or another paper’s measurement, and never on a measurement of our own.

Fix the width first, then read off the count. Take a 95 percent Wilson interval on a single reported rate, at the worst case of $\hat{p} = 0.5$. Writing the half-width of (1) as

$$w(n, \hat{p}) = \frac{z\sqrt{4n\hat{p}(1-\hat{p}) + z^2}}{2(n + z^2)}, \quad (1)$$

the count a target width w_0 demands is $n_{\min}(w_0) = \min\{n : w(n, 0.5) \leq w_0\}$, which gives

half-width w_0 (points)	20	15	10	5
$n_{\min}$ (trials)	21	39	93	381

Ten points is the coarsest width at which a single rate is worth printing, so the absolute-rate minimum is 93, rounded to 100. At 100 trials a reported 80 percent has an interval of 71 to 87 percent, and a reported 50 percent has 40 to 60. A comparison is a different question and a harder one: two rates each carrying ± 10 points do not resolve a 10-point difference between them, because the difference’s standard error is larger by a factor of $\sqrt{2}$, so the width argument sets a floor on what is worth reporting and not on what can be compared.

For the A/B comparison the relevant calculation is power, and the design is paired. Table IV matches initial conditions by image overlay and interleaves the two policies in one session, so the unit is a matched pair and the count follows McNemar, which depends on the discordance rate. The share of initial conditions on which the two policies disagree, and not on the two rates alone. Write $\delta = |p_1 - p_2|$ for the gap between the two success rates, π_d for that discordance rate, $z_{1-\alpha/2} = 1.96$ at $\alpha = 0.05$ and $z_{1-\beta} = 0.84$ at 80 percent power; then the number of matched pairs is

$$n_{\text{pairs}} = \frac{[z_{1-\alpha/2}\sqrt{\pi_d} + z_{1-\beta}\sqrt{\pi_d - \delta^2}]^2}{\delta^2}. \quad (2)$$

To separate 50 from 70 percent, which is $\delta = 0.20$, at $\alpha = 0.05$ with 80 percent power, (2) asks per arm for

discordance π_d	0.2	0.3	0.4	0.5
n_{pairs}	37	57	77	96

The protocol assumes 0.3 and asks for 57, and states the sensitivity rather than hiding it, because 0.5 is the discordance the same two rates produce when the pairing buys nothing, and at that value the paired count returns to the 93 per arm an unpaired test would need. The saving from pairing is real but smaller than the pair counts suggest, since a pair costs two rollouts: 57 pairs is 114 rollouts against 186. An earlier version of this section quoted 93, 169 and 387 per arm for gaps of 20, 15 and 10 points, which are correct for independent arms and are the wrong test for this protocol. That 93 was also the same integer as the half-width calculation in the paragraph above,

which is a coincidence of the worst-case arithmetic and not a second derivation of the same number.

One hundred is a cap and not a bill, because on a graded score a sequential test reached its decision in 12 to 36 paired hardware trials in [52]. At 30 rollouts a continuous score already carries a half-width of ± 0.36 standard deviations, which is why a graded score can stop where a binary one cannot. A cell that stops early does not report a Wilson interval. Optional stopping breaks the coverage of a fixed- n interval, which is the reason Snyder et al. [52] and Badithela et al. [53] use anytime-valid betting intervals rather than Wilson, so Table IV asks a cell run to a fixed 100 for a Wilson interval and a cell stopped early for a confidence sequence, and never for both. Intervals are also marginal rather than simultaneous. For example, evaluating twelve methods on seven axes produces 84 intervals. At 84 independent 95 percent intervals, four excursions are expected by construction, so a paper comparing k policies on m tasks corrects its $k(k-1)/2$ pairwise tests to a global 95 percent level, as the TRI LBM Team [54] does, or says its intervals are not simultaneous.

Simulation is cheap, so simulated cells take 200 episodes, giving a 6.9-point half-width. Perturbation axes are screened rather than certified, and 40 per axis buys a 15-point half-width on each axis’s own absolute rate, which is enough to rank the axes and pick the two worst for hardware. It is not enough for the ratio to the anchor that an earlier draft asked each cell to report. At 40 trials in each arm, a fall from a 0.50 anchor to 0.30 is a ratio of 0.60 with a 95 percent interval of 0.34 to 1.06, which contains 1: at the screening count you cannot establish that the perturbation hurt at all. Certifying that same drop takes 101 per arm at 80 percent power, or about 50 for an interval that merely excludes 1, so Table IV now asks for absolute rates with their own intervals at 40, reports the ratio without an interval, and prescribes 101 before any claim that a named axis hurt.

For unseen objects the resampling unit is the object and not the trial, so 20 objects at 5 trials each gives 100 trials and an object-level half-width near 20 points. That 20 points is the Wilson width at $n = 20$ and it treats each object’s outcome as a single Bernoulli draw, which the five within-object trials are not. It is the right order of magnitude and the assumption belongs in the cell. A 10-point claim about an object distribution needs about 93 objects. Seven of the 39 rows that state an unseen count reach that: 225 in [55], 241 in [34] and [7], 360 in [35], 500 in [56], 2029 in [57] and 503409 in [31]. For a continuous score the half-width is 1.96 standard deviations over the square root of the count, $1.96 s/\sqrt{n}$, so 100 rollouts give ± 0.20 standard deviations, and the unit is the rollout because frames within one are correlated.

The transfer axis is the one where 100 is least defensible. On 100 matched pairs a measured correlation of 0.70 carries a Fisher-z interval of 0.58 to 0.79, a half-width of about 0.10 that 130 pairs would be needed to guarantee. That is enough to establish that a simulator tracks reality at all, and it is not enough to separate [53]’s useful regime from its marginal one, since those differ by about 0.11 in correlation. Both limits come within 0.05 of the estimate only at about 457 pairs. Table IV states which of the two decisions each count supports rather

than leaving a reader to assume the larger one.

REFERENCES

- [1] Nagabandi *et al.*, “Deep Dynamics Models for Learning Dexterous Manipulation,” in *CoRL*, 2019.
- [2] Qin *et al.*, “DexPoint: Generalizable Point Cloud Reinforcement Learning for Sim-to-Real Dexterous Manipulation,” in *CoRL*, 2022.
- [3] Handa *et al.*, “DeXtreme: Transfer of Agile In-hand Manipulation from Simulation to Reality,” in *ICRA 2023*, 2022.
- [4] Chen *et al.*, “Visual Dexterity: In-Hand Reorientation of Novel and Complex Object Shapes,” *Science Robotics* 2023, 2022.
- [5] Petrenko *et al.*, “DexPBT: Scaling up Dexterous Manipulation for Hand-Arm Systems with Population Based Training,” in *RSS*, 2023.
- [6] Wang *et al.*, “PhysHOI: Physics-Based Imitation of Dynamic Human-Object Interaction,” arXiv:2312.04393, 2023.
- [7] Xu *et al.*, “UniDexGrasp: Universal Robotic Dexterous Grasping via Learning Diverse Proposal Generation and Goal-Conditioned Policy,” in *CVPR*, 2023.
- [8] Wang *et al.*, “Lessons from Learning to Spin ‘Pens’,” in *CoRL*, 2024.
- [9] Qian *et al.*, “PianoMime: Learning a Generalist, Dexterous Piano Player from Internet Demonstrations,” in *CoRL*, 2024.
- [10] Zhao *et al.*, “Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware,” in *RSS*, 2023.
- [11] Ze *et al.*, “3D Diffusion Policy: Generalizable Visuomotor Policy Learning via Simple 3D Representations,” in *RSS*, 2024.
- [12] He *et al.*, “OmniH2O: Universal and Dexterous Human-to-Humanoid Whole-Body Teleoperation and Learning,” in *CoRL*, 2024.
- [13] Mandi *et al.*, “DexMachina: Functional Retargeting for Bimanual Dexterous Manipulation,” arXiv:2505.24853, 2025.
- [14] Qi *et al.*, “In-Hand Object Rotation via Rapid Motor Adaptation,” in *CoRL*, 2022.
- [15] Black *et al.*, “pi0: A Vision-Language-Action Flow Model for General Robot Control,” arXiv:2410.24164, 2024.
- [16] NVIDIA, “GR00T N1.6: An Improved Open Foundation Model for Generalist Humanoid Robots,” https://research.nvidia.com/labs/gear/gr00t-n1_6/, 2025, company press release, December 2025, research.nvidia.com, fetched 2026-09-17.
- [17] —, “GR00T N1: An Open Foundation Model for Generalist Humanoid Robots,” arXiv:2503.14734, 2025.
- [18] Yang *et al.*, “AsymDex: Asymmetry and Relative Coordinates for RL-based Bimanual Dexterity,” in *RSS 2025 WCBM workshop (spotlight)*, 2024.
- [19] Jiang *et al.*, “DexMimicGen: Automated Data Generation for Bimanual Dexterous Manipulation via Imitation Learning,” in *ICRA 2025*, 2024.
- [20] Ma *et al.*, “DrEureka: Language Model Guided Sim-To-Real Transfer,” in *RSS*, 2024.
- [21] Cheng *et al.*, “Open-TeleVision: Teleoperation with Immersive Active Visual Feedback,” in *CoRL*, 2024.
- [22] Luo *et al.*, “Being-H0: Vision-Language-Action Pretraining from Large-Scale Human Videos,” arXiv:2507.15597, 2025.
- [23] Physical Intelligence, “pi0.5: a Vision-Language-Action Model with Open-World Generalization,” arXiv:2504.16054, 2025.
- [24] —, “pi*0.6: a VLA That Learns From Experience,” arXiv:2511.14759, 2025.
- [25] Luo *et al.*, “Being-H0.5: Scaling Human-Centric Robot Learning for Cross-Embodiment Generalization,” arXiv:2601.12993, 2026.
- [26] Zhang *et al.*, “ArtiGrasp: Physically Plausible Synthesis of Bi-Manual Dexterous Grasping and Articulation,” in *3DV 2024*, 2023.
- [27] Chi *et al.*, “Diffusion Policy: Visuomotor Policy Learning via Action Diffusion,” in *RSS*, 2023.
- [28] Ma *et al.*, “Eureka: Human-Level Reward Design via Coding Large Language Models,” in *ICLR 2024*, 2023.
- [29] Wan *et al.*, “UniDexGrasp++: Improving Dexterous Grasping Policy Learning via Geometry-aware Curriculum and Iterative Generalist-Specialist Learning,” in *ICCV*, 2023.
- [30] Zhou *et al.*, “Learning Diverse Bimanual Dexterous Manipulation Skills from Human Demonstrations,” in *AAAI 2026 (oral)*, 2024.
- [31] Zhang *et al.*, “GraspXL: Generating Grasping Motions for Diverse Objects at Scale,” in *ECCV*, 2024.
- [32] Fu *et al.*, “HumanPlus: Humanoid Shadowing and Imitation from Humans,” in *CoRL*, 2024.
- [33] Luo *et al.*, “Omnigrasp: Grasping Diverse Objects with Simulated Humanoids,” in *NeurIPS*, 2024.
- [34] Huang *et al.*, “Efficient Residual Learning with Mixture-of-Experts for Universal Dexterous Grasping,” in *ICLR 2025*, 2024.
- [35] Zhong *et al.*, “DexGraspVLA: A Vision-Language-Action Framework Towards General Dexterous Grasping,” arXiv:2502.20900, 2025.
- [36] Liu *et al.*, “DexTrack: Towards Generalizable Neural Tracking Control for Dexterous Manipulation from Human References,” in *ICLR*, 2025.
- [37] Wen *et al.*, “DexVLA: Vision-Language Model with Plug-In Diffusion Expert for General Robot Control,” arXiv:2502.05855, 2025.
- [38] Li *et al.*, “ManipTrans: Efficient Dexterous Bimanual Manipulation Transfer via Residual Learning,” in *CVPR*, 2025.
- [39] An *et al.*, “Dexterous Manipulation through Imitation Learning: A Survey,” arXiv:2504.03515, 2025.
- [40] Welte *et al.*, “Interactive Imitation Learning for Dexterous Robotic Manipulation: Challenges and Perspectives – A Survey,” *Frontiers in Robotics and AI*, 2025.
- [41] Zhao *et al.*, “Sim-to-Real Transfer in Deep Reinforcement Learning for Robotics: a Survey,” in *IEEE SSCI*, 2020.
- [42] Firoozi *et al.*, “Foundation Models in Robotics: Applications, Challenges, and the Future,” *IJRR*, 2023.
- [43] Bai *et al.*, “Towards a Unified Understanding of Robot Manipulation: A Comprehensive Survey,” arXiv:2510.10903, 2025.
- [44] Zhao *et al.*, “Towards Robotic Dexterous Hand Intelligence: A Survey,” arXiv:2605.13925, 2026.
- [45] Kaup *et al.*, “A Review of Nine Physics Engines for Reinforcement Learning Research,” arXiv:2407.08590, 2024.
- [46] Okamura, Smith, and Cutkosky, “An overview of dexterous manipulation,” in *ICRA*, 2000, full text not obtained; cited by metadata only.
- [47] Piazza *et al.*, “A Century of Robotic Hands,” *Annual Review of Control, Robotics, and Autonomous Systems*, 2019, full text not obtained; cited by metadata only.
- [48] Roa and Suárez, “Grasp quality measures: review and performance,” *Autonomous Robots*, 2015, full text not obtained; cited by metadata only.
- [49] Bicchi, “Hands for dexterous manipulation and robust grasping: a difficult road toward simplicity,” *IEEE Transactions on Robotics and Automation*, 2000, full text not obtained; cited by metadata only.
- [50] Ma and Dollar, “On dexterity and dexterous manipulation,” in *ICAR*, 2011.
- [51] Bicchi, “Robotic Grasping and Manipulation (book chapter),” in *RAM-SETE: Articulated and Mobile Robotics, Springer LNCIS*. Springer, 2001.
- [52] Snyder *et al.*, “Beyond Binary Success: Sample-Efficient and Statistically Rigorous Robot Policy Comparison,” arXiv:2603.13616, 2026.
- [53] Badithela *et al.*, “Reliable and Scalable Robot Policy Evaluation with Imperfect Simulators,” arXiv:2510.04354, 2025.
- [54] TRI LBM Team, “A Careful Examination of Large Behavior Models for Multitask Dexterous Manipulation,” arXiv:2507.05331, 2025.
- [55] Shao *et al.*, “Bimanual Grasp Synthesis for Dexterous Robot Hands,” *IEEE RA-L*, 2024.
- [56] Qin *et al.*, “DexMV: Imitation Learning for Dexterous Manipulation from Human Videos,” in *ECCV 2022*, 2021.
- [57] Chen *et al.*, “ClutterDexGrasp: A Sim-to-Real System for General Dexterous Grasping in Cluttered Scenes,” in *CoRL*, 2025.